

```
var b = w.scrollTop();
var o = t.offset();
var x = o.left;
```

DOCUMENTACIÓN DEL PROYECTO

INGENIERÍA DEL SOFTWARE II

Sergio Mirado Tiemblo
Alejandro Arnanz Fernández
Azucena Fernández Bernal
Zaira Muñoz Hernández



```
//mark the element as visible
t.appeared = true;
```

```
//is this supposed to happen only once?
if (settings.one) {
```

```
    //remove the check
    w.unbind('scroll', check);
    var i = $.inArray(check, $.fn.appear.checks);
    if (i >= 0) $.fn.appear.checks.splice(i, 1);
}
```

```
//trigger the original fn
fn.apply(this, arguments);
```

```
the element
settings.data, modifiedFn);
```

Índice

1. Análisis y Definición del Sistema	4
1.1 Objetivo del proyecto	4
1.2 Actores	4
1.3 Requisitos funcionales.....	4
1.4 Requisitos no funcionales	5
1.5 Casos de uso principales	5
1.6 Reglas de negocio.....	6
1.7 Requisitos de datos	6
2. Metodología y Gestión Ágil (SCRUM)	7
2.1 Enfoque Scrum.....	7
2.2 Estructura del equipo y roles	7
2.3 Artefactos y herramientas	7
2.4 Eventos Scrum	7
2.5 Planificación	7
2.6 Control y seguimiento	8
2.7 Backlog y planificación de Sprints.....	8
3. Planificación y gestión del proyecto	9
3.1 Planificación temporal.....	9
3.2 Diagramas UML	10
3.3 Seguimiento y control de tareas.....	11
3.4 Métricas y revisión de Sprints	11
4 Gestión de la configuración.....	12
4.1 Sistema de Control de Versiones	12
4.2 Control de Peticiones de Cambio	12
4.3 Plan de Gestión de Configuración	12
4.5 Construcción y Distribución del Software.....	12
4.6 Gestión de versiones	13
4.7 Control de archivos en el repositorio (gitignore).....	13
5 Descripción de las Funcionalidades	14

5.1 Introducción	14
5.2 Sistema de Login	14
5.3 Sistema de Registro	14
5.4 Configuración de Usuario (Perfil)	15
5.5 Búsqueda por filtros avanzados	16
5.6 Gestión de Inmuebles (Dar de Alta)	16
5.7 Gestión de Inmuebles (Modificación y Eliminación)	17
5.8 Sistema de Favoritos	18
5.9 Sistema de Disponibilidad (Bloqueo de fechas)	18
5.10 Sistema de Reservas	19
5.11 Sistema de Notificaciones	20
6. Calidad del Software (SonarCloud)	21
7. Pruebas	22
7.1 Introducción y objetivos del plan de pruebas	22
7.2 Alcance y tipo de pruebas	22
7.3 Metodología de diseño de pruebas	23
7.4 Implementación de las pruebas en Junit	24
7.5 Medición de la cobertura con JaCoCo	24
8. Mantenimiento	25
8.1 Enfoque general de mantenimiento	25
8.2 Tipos de mantenimiento contemplados	25
8.3 Gestión del mantenimiento en Scrum	27
8.4 Mantenibilidad del sistema	28
8.5 Conclusión	28

1. Análisis y Definición del Sistema

1.1 Objetivo del proyecto

El proyecto tiene como objetivo desarrollar un sistema similar a Airbnb, donde particulares puedan registrar propiedades para alquilar y otros usuarios puedan realizar reservas. Los usuarios deben registrarse previamente y pueden actuar como inquilinos o propietarios, mientras que el sistema también interactúa con una pasarela de pago y un sistema de notificaciones.

1.2 Actores

- **Visitante (no registrado):** puede buscar inmuebles y aplicar filtros básicos.
- **Usuario registrado:** puede autenticarse, mantener su perfil y actuar como inquilino o propietario.
- **Inquilino:** busca propiedades, añade inmuebles a su lista de deseos, realiza reservas y pagos, y puede aplicar filtros avanzados.
- **Propietario:** registra, modifica y elimina propiedades y gestiona solicitudes de reserva.
- **Pasarela de pago (externa):** procesa pagos y reembolsos mediante tarjeta de crédito, débito o PayPal.
- **Sistema de notificaciones (interno):** envía confirmaciones y avisos a los usuarios sobre reservas, pagos y solicitudes.

1.3 Requisitos funcionales

1. Registro y autenticación

El sistema permitirá registrar usuarios, diferenciando inquilinos y propietarios (un usuario puede ser ambos).

Se podrá iniciar y cerrar sesión, gestionando roles y permisos de cada usuario.

2. Gestión de propiedades

Los propietarios podrán registrar propiedades indicando tipo de inmueble, dirección, precio por noche, disponibilidad, comodidades, política de cancelación y tipo de reserva (inmediata o bajo solicitud).

Podrán modificar o eliminar las propiedades registradas.

3. Búsqueda y exploración

Cualquier usuario podrá buscar propiedades sin registro.

Los filtros básicos incluyen destino, fechas y tipo de inmueble.

Los filtros avanzados permiten buscar por reserva inmediata, comodidades específicas o políticas de cancelación.

Los usuarios registrados podrán guardar propiedades en su lista de deseos.

4. Reservas

Los inquilinos registrados podrán realizar reservas.

Si la propiedad permite reserva inmediata, la reserva se confirma tras el pago.

Si requiere solicitud, el inquilino envía una solicitud con pago retenido; el propietario la acepta o rechaza.

En caso de rechazo, se procesa automáticamente el reembolso y se notifica al inquilino.

Cada reserva se asocia a un pago único y cada solicitud puede generar una reserva confirmada si es aceptada.

5. Lista de deseos

Cada inquilino tiene una lista de deseos única.

Una lista puede contener varias propiedades y un inmueble puede estar en varias listas de deseos.

6. Pagos

Se integrará con una pasarela de pagos para validar transacciones antes de confirmar reservas.

Los reembolsos se procesarán automáticamente cuando corresponda.

7. Notificaciones

El sistema notificará sobre confirmación o rechazo de solicitudes de reserva, pagos realizados, reembolsos procesados y nuevas solicitudes recibidas por los propietarios.

1.4 Requisitos no funcionales

1. **Disponibilidad** 24/7 con tolerancia a fallos.
2. **Seguridad** mediante cifrado de contraseñas, HTTPS y protocolos seguros.
3. **Escalabilidad** para miles de usuarios concurrentes.
4. **Usabilidad** e interfaz intuitiva y accesible.
5. **Multiplataforma**: web y app móvil (iOS/Android).
6. **Tiempo de respuesta** óptimo.
7. **Mantenibilidad**: arquitectura modular y documentación clara.

1.5 Casos de uso principales

- Registro de usuarios (inquilino/propietario).
- Inicio y cierre de sesión.
- Registro, modificación y eliminación de propiedades.
- Búsqueda de propiedades (básica y avanzada).
- Agregar propiedades a la lista de deseos.
- Realizar reservas (inmediata o bajo solicitud).

- Realizar pagos.
- Aceptar o rechazar solicitudes de reserva.
- Procesar reembolsos.
- Enviar notificaciones a usuarios.

1.6 Reglas de negocio

- Solo los propietarios registrados pueden registrar propiedades.
- Cada inmueble pertenece a un único propietario; un propietario puede tener varias propiedades.
- No se podrán reservar fechas ya ocupadas.
- Una reserva debe estar asociada a una política de cancelación.
- Los pagos se completan antes de confirmar reservas o solicitudes.
- Los reembolsos se gestionan automáticamente.
- Los usuarios deben aceptar los términos y condiciones al registrarse.
- Las notificaciones a los propietarios son inmediatas ante nuevas solicitudes

1.7 Requisitos de datos

Usuarios: id, nombre, email, contraseña, rol, datos de pago; cada inquilino tiene una lista de deseos.

Propiedades: id, propietario, ubicación, tipo, precio, disponibilidad, comodidades, política de cancelación, tipo de reserva.

Reservas: id, propiedad, inquilino, fechas, estado (pendiente, confirmada, rechazada, cancelada).

Pagos: id, reserva, monto, método, estado (exitoso, rechazado, reembolsado).

Lista de deseos: id usuario, id propiedad.

Notificaciones: id, destinatario, tipo (reserva, pago, reembolso), estado (pendiente, enviada).

Solicitudes de reserva: id, inmueble, inquilino, estado, reserva asociada si fue aceptada.

Disponibilidad: id, inmueble, rango de fechas.

2. Metodología y Gestión Ágil (SCRUM)

2.1 Enfoque Scrum

El desarrollo del sistema se gestionará mediante la metodología ágil **Scrum**, que promueve la entrega incremental del producto, la colaboración continua y la adaptación al cambio. El equipo trabajará en iteraciones cortas denominadas **Sprints**, con entregables funcionales y revisiones periódicas.

2.2 Estructura del equipo y roles

- **Scrum Master:** este rol lo desempeña Azucena Fernández, facilita las reuniones, elimina impedimentos y asegura la correcta aplicación de Scrum.
- **Product Owner:** define las prioridades, gestiona el Product Backlog y vela por el valor del producto.
- **Equipo de desarrollo (Developers):** formado por Alejandro Arnanz, Sergio Mirado y Zaira Muñoz, diseña, implementa, prueba y documenta las funcionalidades. Cada miembro asume responsabilidades específicas según su área (frontend, backend, documentación o testing), pero colaborando de forma transversal.

2.3 Artefactos y herramientas

Los principales artefactos Scrum empleados serán:

- **Product Backlog:** lista priorizada de funcionalidades y tareas.
 - **Sprint Backlog:** conjunto de ítems seleccionados para cada Sprint.
 - **Incremento:** versión funcional del sistema entregada tras cada Sprint.
- Para la gestión se utilizará **GitHub Projects** (tablero Kanban) junto con **Issues** y **Labels** para el control de tareas y seguimiento del avance.

2.4 Eventos Scrum

- **Sprint Planning:** definición de objetivos y tareas del Sprint.
- **Daily Scrum:** breve reunión diaria de seguimiento (puede realizarse asincrónicamente).
- **Sprint Review:** demostración del incremento y revisión de objetivos.
- **Sprint Retrospective:** análisis de mejora continua del proceso.

2.5 Planificación

- El proyecto se desarrollará a lo largo de **X semanas** distribuidas en **n Sprints** de **X días** cada uno.

- Cada Sprint culminará con un incremento funcional verificable
- El **calendario de entregas** incluirá revisiones parciales y una entrega final.

2.6 Control y seguimiento

El control de tareas se realizará mediante **Issues** en GitHub, categorizados por **Labels** que indiquen su tipo y estado (por ejemplo: feature, bug, documentation, in progress, done). Se empleará el **tablero de proyectos (Kanban)** para visualizar el flujo de trabajo y métricas de avance (burn-down chart), revisando en cada Sprint la carga de trabajo completada frente a la planificada.

2.7 Backlog y planificación de Sprints

El **Product Backlog** se construirá a partir de los requisitos funcionales del sistema y se refinará progresivamente.

En cada **Sprint Planning**, el equipo seleccionará los elementos más prioritarios del backlog y los convertirá en **issues detallados**, asignando responsables y estimaciones de esfuerzo. Los Sprints incluirán: desarrollo, revisión de código, testing y documentación.

3. Planificación y gestión del proyecto

3.1 Planificación temporal

Los Sprints del proyecto se han planificado para garantizar la entrega incremental y funcional del sistema. La organización ha sido la siguiente:

- **Sprint 1 (1 semana y 3 días):**

Durante este Sprint se completaron los issues correspondientes al desarrollo inicial, incluyendo:

- Formulario para el registro de usuarios.
- Creación de entidades JPA.
- Implementación de los DAOs.
- Diseño de la base de datos.
- Configuración de la base de datos Derby.
- Configuración de Maven y gestión de dependencias.

- **Sprint 2 (2 semanas y 3 días):**

En este Sprint se finalizaron los siguientes issues:

- Detalle de inmueble.
- Registro de propiedades.
- Creación del catálogo de alojamientos.
- Implementación de la página de inicio.
- Login de usuarios.
- Gestión de errores.
- Diseño de la interfaz web.
- Eliminación de la carpeta database del proyecto para mejorar la organización y control de versiones.

- **Sprint 3 (3 semanas):**

Lo siguientes issues fueron completados en este Sprint:

- Agregar a la Lista de Deseos.
- Gestión del Perfil de Usuario.
- Gestión de Disponibilidad del Inmueble.
- Búsqueda Básica.
- Ver y Gestionar Lista de Deseos.
- Configurar Sonarqube Cloud.

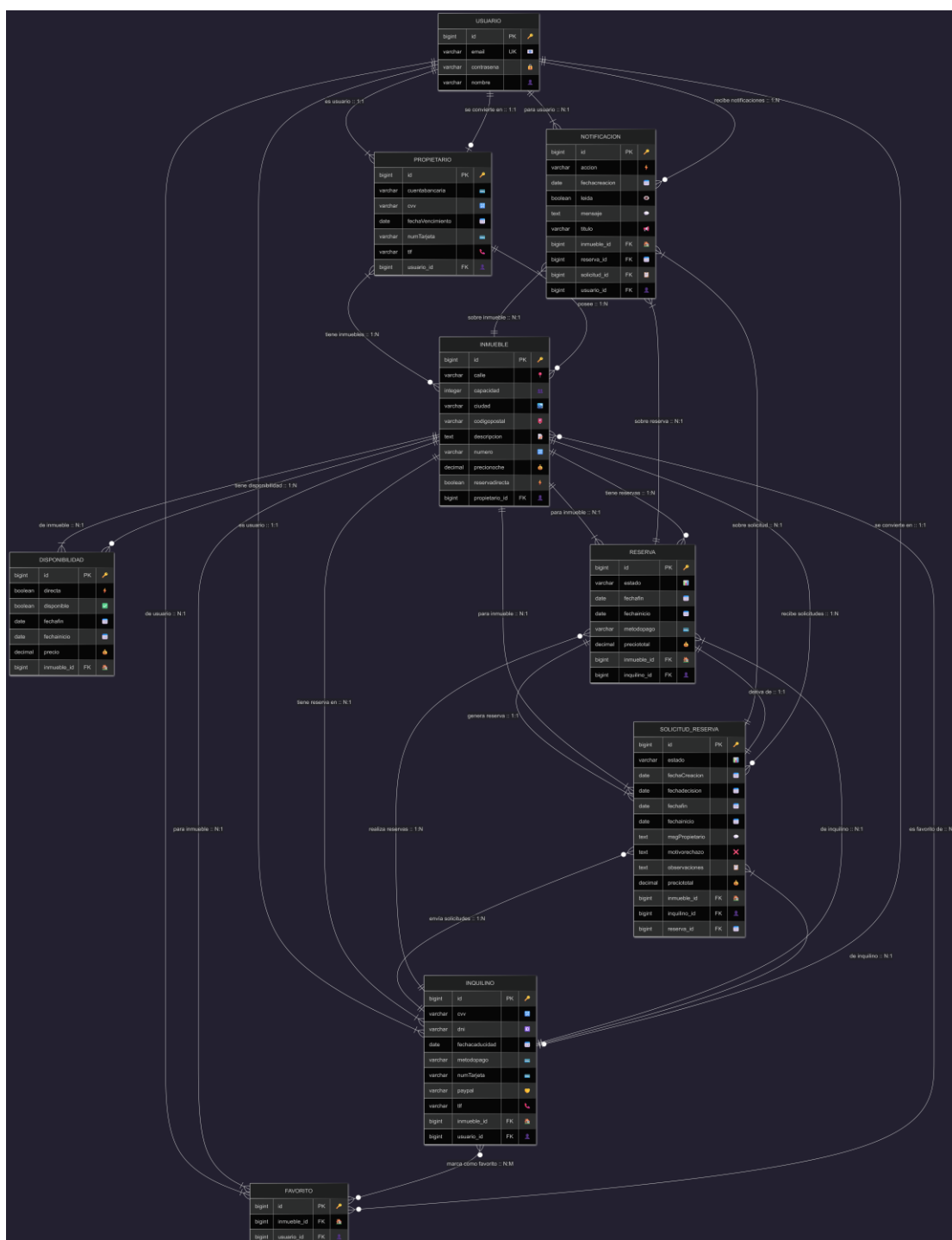
- **Sprint 4 (1 semana y dos días)**

- Modificar o Eliminar Propiedad.
- Gestión estados de reserva.
- Solicitar Reserva (bajo solicitud).
- Reembolsos Automáticos.
- Procesar Pagos.
- Implementar un diseño global.
- Arreglar Issues Sonarqube.

- **Sprint 5 (2 semanas)**
 - Búsqueda Avanzada.
 - Pruebas de Interfaz Web.
 - Pruebas de Integración.
 - Documentación.
 - Validar Restricciones de Negocio.

3.2 Diagramas UML

- Relaciones de la base de datos



3.3 Seguimiento y control de tareas

- **Issues de GitHub:** cada historia de usuario se convierte en un issue, etiquetado según tipo (frontend, backend, database...).
- **Kanban board:** seguimiento de tareas en columnas (Product backlog, sprint backlog...).
- **Burn-down chart:** gráfica de progreso de cada Sprint (historias completadas vs. tiempo).

3.4 Métricas y revisión de Sprints

- Evaluación de avance mediante porcentaje de historias completadas.
- Revisión de objetivos en Sprint Review, ajustes en el Backlog según prioridades.
- Retrospectiva para identificar mejoras en el proceso y corregir desviaciones.

4 Gestión de la configuración

4.1 Sistema de Control de Versiones

- **Sistema utilizado:** Git
- **Repositorio:** GitHub
- **Convenciones básicas:**
 - Rama principal: main
 - Rama de desarrollo: development
 - Ramas de funcionalidades: feature/*
 - Ramas de corrección de errores: hotfix/*
- **Etiquetado de releases:** versión semántica vX.Y.Z

4.2 Control de Peticiones de Cambio

- Cada cambio importante se registra en **GitHub Issues**, indicando:
 - Tipo: feature, bug, documentation...
 - Estado: open, in progress, revision, done
 - Descripción del cambio
- Las peticiones de cambio se priorizan y asignan a miembros del equipo.

4.3 Plan de Gestión de Configuración

- **Elementos de configuración:**
 - Código fuente (src/main/java)
 - Plantillas y recursos (src/main/resources/templates, application.properties)
 - Archivos de build (pom.xml)
- **Procedimiento de cambio:**
 1. Crear una rama nueva (feature/* o bugfix/*)
 2. Implementar el cambio
 3. Abrir Pull Request para revisión
 4. Mergear en develop tras aprobación
 5. Mergear en main solo si pasa pruebas y build

4.5 Construcción y Distribución del Software

- **Herramienta:** Maven
- **Comandos básicos:**
 - mvn clean install → build local
 - mvn spring-boot:run → ejecutar la aplicación
- **Distribución:** se entrega la versión compilada (target/) y el repositorio actualizado.

4.6 Gestión de versiones

El equipo de desarrollo utilizará única y exclusivamente las siguientes versiones:

- **Java:** 21
- **Apache Derby:** 10.16.1.1
- **Apache Maven:** 3.9.11
- **Spring Boot:** 3.5.6

Se recomienda mantener estas versiones consistentes en todos los entornos de desarrollo para asegurar compatibilidad y reproducibilidad del proyecto.

4.7 Control de archivos en el repositorio (gitignore)

Con el fin de garantizar la integridad y portabilidad del proyecto, así como evitar la inclusión de archivos temporales, binarios o específicos del entorno de desarrollo, se ha definido una política de exclusión para Git mediante el archivo **.gitignore**. Esto permite mantener el repositorio limpio y asegurar que solo el código fuente y los recursos necesarios para la construcción y ejecución del proyecto sean compartidos.

- **Archivos compilados y paquetes**
Se excluyen los archivos generados durante la compilación y empaquetado, como binarios y paquetes, evitando subir artefactos que se pueden reconstruir.
- **Archivos de registro**
Se excluyen los archivos de log generados durante la ejecución de la aplicación, incluyendo logs de frameworks específicos como Spring Boot.
- **Directorios de construcción y dependencias**
Se omiten los directorios de construcción y dependencias generados por herramientas de gestión de proyectos como Maven o Gradle, evitando conflictos entre entornos.
- **Archivos y configuraciones de entornos de desarrollo**
Se excluyen los archivos y carpetas generados automáticamente por los distintos IDEs (Eclipse, IntelliJ IDEA, VS Code) que no son necesarios para la construcción del proyecto ni para su colaboración.
- **Archivos temporales del sistema operativo**
Se evitan los archivos generados automáticamente por el sistema operativo, que no aportan información relevante al proyecto.
- **Archivos de entorno local**
Se excluyen las configuraciones locales y variables de entorno que pueden contener información sensible o específica de cada desarrollador.
- **Otros archivos específicos del proyecto**
Se omiten otros archivos generados por el proyecto, como bases de datos embebidas o directorios temporales, que no deben subirse al repositorio.

5 Descripción de las Funcionalidades

5.1 Introducción

En esta sección se describen de manera detallada todas las funcionalidades implementadas en la plataforma web de alojamientos. Cada funcionalidad incluye su objetivo, el comportamiento esperado y la relación con las clases, controladores y servicios implicados.

Esta documentación permite comprender la estructura lógica de la aplicación y asegura una clara trazabilidad entre los requisitos y la implementación técnica.

5.2 Sistema de Login

Descripción general

Ofrece un mecanismo seguro de identificación de usuarios mediante email y contraseña. Incluye validaciones, manejo de sesiones y control de acceso para diferenciar entre usuarios normales y propietarios.

Objetivo

Garantizar que solo usuarios autorizados acceden a funcionalidades sensibles del sistema, protegiendo la información personal y evitando accesos indebidos

Comportamiento

- Validación de credenciales.
- Creación de sesión en servidor.
- Restricción automática de rutas protegidas.

Clases implicadas

Capa	Clase
Controlador	LoginController.java
Entidad	Usuario.java
Persistencia	UsuarioDAO.java
Vista	login.html, login-greeting.html, resultLogin.html

5.3 Sistema de Registro

Descripción general

Permite que nuevos usuarios creen una cuenta proporcionando datos básicos. El sistema valida la entrada, comprueba duplicados y almacena de forma segura la información del nuevo usuario.

Objetivo

Ampliar la base de usuarios de la plataforma y permitirles acceder a servicios como reservas, favoritos y notificaciones.

Comportamiento

- Validación de información.
- Creación de usuario en la base de datos.
- Habilitar sistema de login.

Clases implicadas

Capa	Clase
Controlador	GestorUsuarios.java
Entidad	Usuario.java
Persistencia	UsuarioDAO.java
Vista	greeting.html,result.html

5.4 Configuración de Usuario (Perfil)

Descripción general

Esta funcionalidad permite que cada usuario gestione su información personal dentro del sistema, como nombre, apellidos, email, contraseña y otros datos relevantes del perfil. La sección ofrece un entorno sencillo e intuitivo, con la capacidad de modificar información, administrar preferencias y revisar datos esenciales de forma segura.

Objetivo

Proporcionar un espacio donde el usuario pueda mantener su información actualizada, fortaleciendo la confianza en el sistema y garantizando que los datos almacenados son correctos para procesos posteriores como reservas, notificaciones o pagos.

Comportamiento

- Visualización del perfil actual.
- Actualización de información existente.
- Validación de datos modificados

Clases implicadas

Capa	Clase
Controlador	ConfiguracionUsuarioController.java
Entidad	Usuario.java, Propietario.java, Favorito.java
Persistencia	UsuarioDAO.java, PropietariODAO.java, FavoritoDAO.java
Vista	configuracionusuario.html, profile.html

5.5 Búsqueda por filtros avanzados

Descripción general

La aplicación permite al usuario localizar alojamientos de forma eficiente mediante un sistema de filtros avanzados. El usuario puede aplicar criterios como ubicación, rango de precios, número de huéspedes, disponibilidad por fechas o servicios incluidos. El sistema evalúa estos parámetros y devuelve únicamente los resultados que coinciden con la búsqueda, optimizando así la experiencia del usuario y reduciendo tiempos de navegación.

Objetivo

Facilitar al usuario la identificación rápida del alojamiento que mejor se ajusta a sus necesidades específicas, evitando recorridos largos o búsquedas manuales extensas. Esta funcionalidad contribuye directamente a mejorar la usabilidad, precisión en los resultados y satisfacción de la experiencia general.

Comportamiento

- Entrada de parámetros desde formulario.
- Procesamiento de filtros dinámicos.
- Consulta a base de datos.

Clases implicadas

Capa

Controlador

Clase

BusquedaController.java, GestorDisponibilidad.java, PropiedadController.java, GestorNotificaciones.java

Entidad

Inmueble.java, Reserva.java, Notificacion.java

Persistencia

InmuebleDAO.java, ReservaDAO.java,

Vista

index.html, lista-propiedades.html

5.6 Gestión de Inmuebles (Dar de Alta)

Descripción general

Los propietarios pueden registrar nuevos inmuebles en la plataforma mediante un formulario detallado que recoge datos como ubicación, precio, servicios y descripción. El sistema valida la información y guarda el inmueble en la base de datos para incorporarlo automáticamente al catálogo público.

Objetivo

Permitir que los propietarios amplíen su oferta de alojamientos y que el sistema mantenga actualizado su inventario. Esta funcionalidad es fundamental para asegurar un flujo constante de nuevos inmuebles y mejorar la disponibilidad para los viajeros.

Comportamiento

- Formulario de creación.
- Validaciones.
- Asignación del propietario.
- Guardado en base de datos.

Clases implicadas

Capa	Clase
Controlador	GestorInmuebles.java
Entidad	Inmueble.java, Propietario.java, Usuario.java
Persistencia	InmuebleDAO.java, UsuarioDAO.java, PropietarioDAO.java
Vista	registro-propiedad.html, resultado-propiedad.html

5.7 Gestión de Inmuebles (Modificación y Eliminación)

Descripción general

Los propietarios pueden modificar cualquier aspecto de sus inmuebles ya registrados. Asimismo, cuentan con la capacidad de eliminar una propiedad, teniendo el sistema mecanismos para retirar también datos dependientes como reservas futuras, solicitudes o favoritos. Estas acciones se realizan de manera segura para no comprometer la integridad de la base de datos.

Objetivo

Garantizar que los inmuebles estén siempre correctamente actualizados y que los propietarios mantengan el control total sobre su catálogo. También permite asegurar que las reservas o solicitudes relacionadas sean gestionadas adecuadamente ante cualquier cambio

Comportamiento

- Validación de propiedad del inmueble.
- Actualización de datos.
- Eliminación con control de integridad:
 - Cancelación de reservas en curso.
 - Aviso a los inquilinos afectados.
 - Eliminación de favoritos y solicitudes asociadas.

Clases implicadas

Capa	Clase
Controlador	GestorInmuebles.java, DetallesInmuebleController.java, MisPropiedadesController.java,
Entidad	Inmueble.java, Propietario.java, Usuario.java
Persistencia	InmuebleDAO.java, UsuarioDAO.java, PropietarioDAO.java

Vista

editar-propiedad.html, modificar-propiedad.html

5.8 Sistema de Favoritos

Descripción general

Los usuarios pueden marcar alojamientos como favoritos para guardarlos y revisarlos posteriormente. Estos inmuebles quedan vinculados al perfil del usuario, permitiéndole consultar su lista personal en cualquier momento.

Objetivo

Mejorar la experiencia de navegación permitiendo que los usuarios guarden alojamientos de interés sin necesidad de buscarlos nuevamente. Esta funcionalidad fomenta la permanencia en la plataforma y agiliza futuras reservas.

Comportamiento

- Añadir o quitar de favoritos.
- Visualización de lista de favoritos.
- Eliminación automática si el inmueble es borrado.

Clases implicadas

Capa

Controlador

Entidad

Persistencia

Vista

Clase

FavoritoController.java

Favorito.java, Inmueble.java

InmuebleDAO.java, FavoritoDAO.java

lista-deseos.html

5.9 Sistema de Disponibilidad (Bloqueo de fechas)

Descripción general

El sistema bloquea automáticamente las fechas ocupadas por reservas confirmadas para evitar solapamientos. Esta gestión se actualiza en tiempo real y se integra con los filtros de búsqueda para no mostrar fechas incorrectas.

Objetivo

Asegurar que un alojamiento no pueda ser reservado más de una vez en la misma fecha. Esto mantiene la integridad del sistema de reservas y evita conflictos entre usuarios.

Comportamiento

- Consulta de reservas existentes.
- Bloqueo automático de fechas elegidas por otros usuarios.
- Visualización en calendario.

Clases implicadas

Capa	Clase
Controlador	GestorDisponibilidad.java
Entidad	Disponibilidad.java, Reserva.java, Inmueble.java, SolicitudReserva.java
Persistencia	DisponibilidadDAO.java, ReservaDAO.java, InmuebleDAO.java, SolicitudReservaDAO.java Vista

5.10 Sistema de Reservas

Descripción general

Los usuarios pueden reservar inmuebles directamente si el propietario acepta reservas automáticas, o pueden enviar una solicitud que el propietario debe aprobar. El sistema gestiona los estados de cada reserva (pendiente, aceptada, rechazada, cancelada), combinando lógica de negocio, validaciones y notificaciones automáticas.

Objetivo

Ofrecer un proceso flexible y adaptable a las necesidades de cada propietario, manteniendo una experiencia clara y organizada para los usuarios interesados en alojarse en un inmueble.

Comportamiento

- Selección de fechas.
- Validación de disponibilidad.
- Creación de solicitud o reserva confirmada.
- Confirmación o rechazo por parte del Propietario para las solicitudes.
- Actualización del estado (pendiente, aceptada, cancelada...

Clases implicadas

Capa	Clase
Controlador	GestorReservas.java, ReservasController.java, SolicitudesController.java, GestorDisponibilidad.java, GestorNotificaciones.java
Entidad	Reserva.java, SolicitudReserva.java, Inmueble.java, Inquilino.java, Disponibilidad.java, Propietario.java
Persistencia	ReservaDAO.java, SolicitudReservaDAO.java, InmuebleDAO.java, InquilinoDAO.java, DisponibilidadDAO.java, PropietarioDAO.java

Vista

reserva-inmueble.html, confirmacion-reserva.html

5.11 Sistema de Notificaciones

Descripción general

La plataforma cuenta con un sistema interno que envía notificaciones a usuarios e inquilinos cuando se producen eventos importantes, como la confirmación o cancelación de una reserva, solicitudes nuevas o eliminación de inmuebles. Las notificaciones quedan disponibles dentro del panel del usuario. En el caso de los Propietarios, también recibirán notificaciones respecto a solicitudes pendientes, que podrán aceptar o rechazar.

Objetivo

Mantener a los usuarios informados en todo momento sobre el estado de sus operaciones, evitando malentendidos y aportando transparencia al sistema y facilitándole a los propietarios la gestión de reserva de sus propiedades.

Comportamiento

- Generación de notificación.
- Envío a la bandeja interna del usuario.
- Lectura desde el panel de notificaciones.
- Mostrar lista paginada o completa.
- Marcar como leída.
- Borrado manual de notificaciones antiguas.

Clases implicadas

Capa

Controlador

Clase

NotificacionesController.java, GestorNotificaciones.java,
GestorReservas.java

Entidad

Notificacion.java

Persistencia

NotificacionDAO.java

Vista

notificaciones.html, notificacionesUsuario.html

6. Calidad del Software (SonarCloud)

En primer lugar, configuramos la versión en línea y gratuita de SonarQube con el objetivo de evaluar la calidad de nuestro código y detectar los problemas (issues) asociados a este. El proceso de configuración resultó algo complejo, principalmente debido a las limitaciones propias de la edición gratuita, que no ofrecía todas las funcionalidades requeridas por el profesor. Esto nos obligó a explorar alternativas y a comprender mejor el alcance real de las restricciones de la herramienta.

Posteriormente, el profesor habilitó un servidor dedicado de SonarQube en la dirección IP: <http://172.20.48.132:9000/>, indicándonos que debíamos conectarnos mediante la red de la Universidad o, en su defecto, utilizar una VPN para garantizar el acceso. Gracias a ello, pudimos ejecutar un análisis más completo de nuestro código y comenzar a resolver los issues detectados, simultáneamente configurando la Quality Gate necesaria para cumplir con los estándares exigidos en la asignatura.

Entre los issues más relevantes encontramos aquellos vinculados a los controladores. En muchos casos, SonarQube desaconsejaba el uso de la anotación “@Autowired” directamente sobre los atributos de las clases por motivos de mantenibilidad. En su lugar, recomendaba aplicar la inyección de dependencias mediante un constructor que incluyera los atributos requeridos, favoreciendo así un diseño más estructurado y fácil de probar. Otro issue destacado surgió en el módulo de login, donde se identificó un problema de seguridad catalogado con riesgo alto. Este problema podía dar lugar a acciones maliciosas por parte de un usuario, por lo que procedimos a restringir los datos de entrada para garantizar un nivel adecuado de seguridad dentro del proyecto.

Por último, cabe resaltar que SonarQube ofrece funcionalidades especialmente valiosas, como la indicación precisa del fragmento de código que debe modificarse y la recomendación explícita de los cambios a aplicar. Esta característica, junto con la estimación del tiempo necesario para resolver cada issue, nos permitió priorizar tareas y gestionar de manera más eficiente el proceso de refactorización. En conjunto, estas herramientas han contribuido significativamente a mejorar la calidad del código y a adoptar buenas prácticas de desarrollo.

7. Pruebas

7.1 Introducción y objetivos del plan de pruebas

El objetivo principal del plan de pruebas ha sido validar el correcto funcionamiento de los distintos componentes del sistema, poniendo especial énfasis en la lógica de negocio asociada a la gestión de usuarios, inmuebles, reservas, disponibilidad, pagos y notificaciones. Para ello, se ha definido y ejecutado un conjunto de pruebas unitarias diseñadas siguiendo la metodología vista en la asignatura, evitando pruebas triviales y centrándonos en escenarios representativos, casos límite y combinaciones relevantes de valores de entrada.

7.2 Alcance y tipo de pruebas

7.2.1 Alcance de las pruebas

El alcance de las pruebas abarca principalmente las siguientes capas del sistema:

- **Capa de persistencia:** repositorios (DAOs) basados en Spring Data JPA, responsables del acceso a datos.
- **Capa de controladores:** controladores web encargados de gestionar las peticiones HTTP, la sesión de usuario y la interacción con la lógica de negocio.
- **Lógica de negocio integrada en servicios:** clases gestoras que encapsulan reglas de negocio relevantes, especialmente en los módulos de reservas, disponibilidad y notificaciones.

7.2.2 Tipo de pruebas realizadas

De acuerdo con el enunciado del proyecto, se han implementado **pruebas unitarias**, entendidas como pruebas que verifican el comportamiento de una unidad concreta de software de forma aislada o con dependencias controladas.

En particular, se han utilizado los siguientes enfoques:

- **Pruebas unitarias de persistencia**, empleando `@DataJpaTest`, que permiten verificar el correcto funcionamiento de los métodos de los DAOs sobre una base de datos embebida de pruebas.
- **Pruebas unitarias de controladores**, utilizando `@WebMvcTest` y `MockMvc`, simulando peticiones HTTP y controlando las dependencias mediante mocks cuando ha sido necesario.
- **Pruebas orientadas a lógica de negocio**, centradas en validar reglas y flujos críticos (por ejemplo, validación de fechas, creación de reservas, gestión de solicitudes o notificaciones).

Este enfoque permite validar tanto el comportamiento interno como la correcta orquestación entre componentes, manteniendo el carácter unitario de las pruebas.

7.3 Metodología de diseño de pruebas

Siguiendo las indicaciones del profesorado y la metodología vista en la parte teórica de la asignatura, el diseño de las pruebas **no se ha limitado a comprobaciones triviales (como verificar getters y setters), sino que se ha basado en técnicas sistemáticas de diseño de casos de prueba.**

7.3.1 Partición en clases de equivalencia

Para cada funcionalidad relevante, se han identificado **clases de equivalencia** de los datos de entrada, diferenciando entre:

- Valores válidos.
- Valores inválidos.
- Valores límite.

Por ejemplo:

- Fechas de reserva válidas frente a fechas en el pasado o rangos inválidos.
- Usuarios autenticados frente a usuarios no autenticados.
- Inmuebles existentes frente a identificadores inexistentes.
- Estados válidos de una reserva o solicitud frente a estados no permitidos en una determinada operación.

De cada clase de equivalencia se han seleccionado valores representativos para construir los casos de prueba, asegurando que el comportamiento del sistema es correcto en cada uno de estos escenarios.

7.3.2 Cobertura de combinaciones de valores

En aquellas funcionalidades donde intervienen múltiples parámetros o condiciones, se han definido casos de prueba que cubren **combinaciones relevantes de valores**, especialmente cuando estas combinaciones afectan al flujo de ejecución del sistema.

Algunos ejemplos de combinaciones consideradas son:

- Usuario autenticado/no autenticado combinado con la existencia o no de un inmueble.
- Tipo de reserva (inmediata o bajo solicitud) combinado con la disponibilidad de fechas.
- Estado de una solicitud combinado con la acción del propietario (aprobar o rechazar).
- Existencia de reservas futuras combinada con la eliminación de un inmueble.

Este enfoque permite cubrir diferentes ramas del código y validar correctamente las reglas de negocio más complejas.

7.4 Implementación de las pruebas en JUnit

7.4.1 Herramientas utilizadas

Las pruebas se han implementado utilizando las siguientes herramientas y tecnologías:

- **JUnit 5 (JUnit Jupiter)** como framework de pruebas.
- **Spring Boot Test**, para facilitar la carga del contexto de pruebas.
- **MockMvc**, para simular peticiones HTTP en pruebas de controladores.
- **Mockito**, para la creación de mocks y el control de dependencias.
- **Apache Derby embebido**, como base de datos de pruebas en los tests de persistencia.

7.4.2 Pruebas de persistencia

Las pruebas de la capa de persistencia se han implementado con `@DataJpaTest`, lo que permite validar el comportamiento real de los repositorios JPA sobre una base de datos en memoria.

En estas pruebas se ha verificado, entre otros aspectos:

- Correcta inserción y recuperación de entidades.
- Funcionamiento de métodos personalizados de consulta.
- Comportamiento ante la inexistencia de datos.
- Cumplimiento de restricciones de integridad definidas en las entidades.

Este enfoque ha permitido alcanzar una cobertura muy elevada en la capa de persistencia y asegurar la fiabilidad del acceso a datos.

7.4.3 Pruebas de controladores

Las pruebas de controladores se han implementado mediante `@WebMvcTest`, simulando el comportamiento del sistema ante peticiones HTTP reales.

En estas pruebas se han validado aspectos como:

- Acceso correcto o denegado a funcionalidades según el estado de la sesión.
- Redirecciones esperadas en función de la lógica de negocio.
- Gestión de errores y validaciones de entrada.
- Interacción entre controladores y servicios.

Para evitar dependencias con la capa de presentación (plantillas Thymeleaf), se ha configurado un resolutor de vistas de pruebas, permitiendo centrar las pruebas en la lógica del controlador y no en el renderizado de vistas.

7.5 Medición de la cobertura con JaCoCo

Con el objetivo de evaluar de forma objetiva el alcance de las pruebas implementadas, se ha integrado la herramienta **JaCoCo** en el ciclo de construcción del proyecto mediante Maven.

accommodationapp

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
com.mantecasl.accommodationapp.business.controller		74%		55%	117 267	193 843	18 120	0 17
com.mantecasl.accommodationapp.business.entity		83%		92%	39 224	59 357	37 210	0 11
com.mantecasl.accommodationapp		100%		n/a	0 2	0 3	0 2	0 1
com.mantecasl.accommodationapp.business.persistence		100%		n/a	0 1	0 1	0 1	0 1
Total		1,101 of 4,799 77%		134 of 322 58%	156 494	252 1,204	55 333	0 30

Cobertura Jacoco del proyecto

Es importante destacar que la cobertura se ha utilizado como **indicador de apoyo**, priorizando siempre la coherencia entre el diseño de pruebas documentado y la implementación real, tal y como se indica en las directrices del proyecto.

8. Mantenimiento

8.1 Enfoque general de mantenimiento

El mantenimiento del sistema se ha planteado como una actividad continua a lo largo de todo el ciclo de vida del proyecto, no únicamente como una fase posterior al desarrollo. Desde el inicio, la aplicación se ha diseñado siguiendo principios de modularidad, separación por capas y buenas prácticas de desarrollo, con el objetivo de facilitar la corrección de errores, la evolución funcional y la adaptación a nuevos entornos o requisitos.

La estructura del proyecto basada en Spring Boot, junto con el uso de Maven, Git/GitHub y una arquitectura organizada en paquetes (controladores, entidades, persistencia y configuración), permite realizar tareas de mantenimiento de forma controlada, trazable y segura.

8.2 Tipos de mantenimiento contemplados

8.2.1 Mantenimiento correctivo

Uso de GitHub Issues para registrar incidencias y errores detectados durante el desarrollo.

Corrección de problemas funcionales y técnicos en distintos componentes del sistema, especialmente relacionados con la gestión de usuarios, reservas y búsquedas.

Resolución de errores de ejecución y mejora del manejo de excepciones.

Corrección de vulnerabilidades y problemas de calidad identificados por SonarQube, especialmente en la validación de entradas de formularios sensibles como login y registro.

Las correcciones se han integrado en el flujo habitual de desarrollo mediante commits y Pull Requests, garantizando la revisión del código antes de su incorporación a la rama principal.

8.2.2 Mantenimiento preventivo

El mantenimiento preventivo ha sido clave para reducir la deuda técnica y mejorar la calidad del código a largo plazo. Se ha aplicado principalmente mediante:

- Análisis continuo con SonarQube, identificando:
 - Code smells.

- Problemas de mantenibilidad.
 - Riesgos de seguridad.
- Refactorización de código, especialmente en controladores con alta responsabilidad, separando lógica y mejorando la legibilidad.
- Sustitución de la inyección de dependencias mediante @Autowired por inyección por constructor, tal y como recomienda SonarQube, aumentando la testabilidad y robustez del sistema.
- Uso de convenciones claras de nombres, estructura de paquetes y eliminación de código duplicado.

Estas acciones han permitido mantener un código más limpio, comprensible y fácil de extender.

8.2.3 Mantenimiento adaptativo

El mantenimiento adaptativo se centra en la capacidad del sistema para ajustarse a cambios en el entorno tecnológico o en los requisitos externos.

En este proyecto se ha tenido en cuenta mediante:

- Uso de versiones concretas y estables:
 - Java 21
 - Spring Boot 3.5.6
 - Apache Derby 10.16.1.1
 - Maven 3.9.11
- Centralización de la configuración en application.properties.
- Exclusión de bases de datos embebidas y archivos de entorno mediante .gitignore, evitando dependencias del entorno local.
- Arquitectura desacoplada que permitiría, en el futuro, sustituir Apache Derby por otro SGBD o integrar pasarelas de pago reales sin afectar al núcleo del sistema.

8.2.4 Mantenimiento perfectivo

El mantenimiento perfectivo se refiere a la mejora y ampliación de funcionalidades existentes o a la incorporación de nuevas características.

Gracias a la estructura modular del proyecto, sería sencillo:

- Añadir nuevos filtros avanzados en el sistema de búsqueda (PropiedadController, BusquedaController).
- Ampliar el sistema de notificaciones con nuevos tipos de eventos.
- Incorporar nuevas políticas de cancelación.
- Implementar funcionalidades futuras como valoraciones de usuarios o mensajería interna.

La separación clara entre controladores, entidades y DAOs facilita estas ampliaciones sin introducir efectos colaterales significativos.

8.2.5 Papel de las pruebas en el mantenimiento del sistema

Las pruebas implementadas en el proyecto desempeñan un papel fundamental como soporte al mantenimiento del sistema, especialmente en las tareas de mantenimiento correctivo y preventivo. La existencia de pruebas unitarias sobre la capa de persistencia, los controladores y la lógica de negocio permite verificar que las correcciones de errores y las mejoras técnicas no introducen regresiones en funcionalidades ya implementadas.

Durante el mantenimiento correctivo, las pruebas facilitan la validación de los errores corregidos y garantizan que el comportamiento esperado del sistema se mantiene tras aplicar los cambios. Asimismo, en el mantenimiento preventivo, las pruebas proporcionan una base de seguridad que permite realizar refactorizaciones y mejoras de calidad del código con mayor confianza.

En conjunto, el uso de pruebas contribuye directamente a mejorar la mantenibilidad del sistema, reduciendo el riesgo asociado a futuras modificaciones y facilitando la evolución del software a lo largo del tiempo.

8.3 Gestión del mantenimiento en Scrum

El mantenimiento del sistema no se ha abordado como una fase independiente, sino que se ha integrado de forma natural dentro del proceso de desarrollo ágil basado en Scrum adoptado en el proyecto.

Las tareas relacionadas con mantenimiento han surgido principalmente durante el desarrollo de los distintos Sprints y se han gestionado mediante **GitHub Issues**, al igual que el resto de funcionalidades. Estas tareas han incluido correcciones de errores, ajustes técnicos, mejoras de calidad del código y reorganización de elementos del proyecto.

Un ejemplo representativo de este enfoque es la resolución de los **issues detectados por SonarQube**, gestionados como tareas específicas dentro de los Sprints, así como la corrección de errores de manejo de excepciones, configuración del sistema y mejoras estructurales del proyecto (como la reorganización de elementos del repositorio y la eliminación de recursos innecesarios del proyecto).

La priorización de estas tareas se ha realizado junto con el resto de issues del Sprint, atendiendo a su impacto técnico y funcional sobre el sistema. De este modo, el mantenimiento ha formado parte del flujo de trabajo habitual del equipo, garantizando la estabilidad y calidad del software sin interrumpir el desarrollo incremental del producto.

Este enfoque es coherente con los principios de Scrum, donde la mejora continua y la corrección de problemas técnicos se integran dentro del ciclo iterativo de desarrollo.

8.4 Mantenibilidad del sistema

La mantenibilidad del sistema se ve reforzada por los siguientes factores:

- Arquitectura por capas claramente definida:
 - Controladores (controller)
 - Lógica de dominio (entity)
 - Persistencia (persistance)
 - Configuración (config)
- Uso de DAOs independientes para cada entidad.
- Código documentado y estructurado.
- Control de versiones y revisión de código mediante Pull Requests.
- Análisis de calidad con SonarQube y aplicación de estándares de buenas prácticas.

Todo ello contribuye a que el sistema sea fácil de entender, modificar y evolucionar, incluso por desarrolladores que no hayan participado en el desarrollo inicial.

8.5 Conclusión

El mantenimiento del sistema ha sido considerado un aspecto fundamental desde el inicio del proyecto. La combinación de buenas prácticas de desarrollo, metodología ágil, control de versiones y herramientas de calidad ha permitido construir una aplicación robusta, escalable y preparada para evolucionar en el tiempo.

Gracias a esta planificación, el sistema no solo cumple con los requisitos actuales, sino que está preparado para futuras mejoras, correcciones y adaptaciones, garantizando su viabilidad a largo plazo.